et al. (2014b), linearizes the loss function to compute perturbations that maximize the loss while adhering to adversarial constraints, requiring several backpropagation steps:

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(f(x), y))$$

However, gradient-based attacks do not directly apply to text data because 1) the text data space x is discrete instead of continuous, and 2) the adversarial region is hard to define for the text data. In this work, we define two perturbations for optimizing text adversarial examples based on our targeted loss function using gradient information: a) **token-level attack**: random token insertion and gradient-informed replacement using token gradients derived by our novel loss function, b) **embedding-level attack**: imperceptible l_∞ perturbations on question inputs' embedding spaces.

In-context Learning (ICL) with CoTs. Our evaluation framework begins by using a set of labeled instances, each paired with a human-crafted CoT, to prompt LLMs. In ICL, given an unlabeled question $x_p \in \mathcal{X}$ and training examples $(x_i, y_i, r_i)_{i=1}^k$, where $x_i \in \mathcal{X}$ is a question, $y_i \in \mathcal{Y}$ is its answer and $r_i \in \mathcal{R}$ is the corresponding CoT, we generate the most likely answer and CoT for the unlabeled question x_p with model f parameterized by θ:

$$\underset{(y_p, r_p) \in \mathcal{R} \times \mathcal{Y}}{\text{argmax}} \; P_\theta \left((y_p, r_p) \mid (x_i, y_i, r_i)_{i=1}^k, (x_p) \right)$$

We aim to generate the most probable pair of an answer y_p and a Chain-of-Thought r_p from an LLM. We define y_p as a preemptive answer, meaning that the answer is generated before the reasoning. This design choice allows us to rigorously isolate the robustness of the reasoning process via an *Answer-Conditioned strategy*. In standard reasoning-first approaches, the answer is causally downstream of the rationale; as a result, perturbations that degrade the reasoning often propagate to the final prediction. This coupling introduces a confounding factor that obscures the specific phenomenon of "Right for the Wrong Reasons". By anchoring the model's predicted answer first, we explicitly decouple decision-making from rationalization. This enables us to control the decision boundary and stress-test the reasoning in isolation, ensuring that any observed fragility reflects properties of the reasoning structure itself rather than being a consequence of answer changes.

4 MATCHA: Misaligned Answer and Thought CHAin

In this section, we introduce a novel evaluation framework MATCHA that constructs perturbations that selectively degrade CoT quality while preserving the generated correct answer. Unlike traditional perturbations that target model predictions directly, our goal is more nuanced: to expose discrepancies between correct answers and faulty reasoning, thereby revealing hidden failure modes in the reasoning process. Achieving this demands a carefully crafted optimization strategy. To be more specific, we present two complementary methods: **token-level** perturbations, which manipulate the input at the discrete level, and **embedding-level** perturbations, which operate directly in the continuous representation space. Each method presents its challenges and reveals different aspects of the model's vulnerability.

4.1 Answer-Conditioned Probing

A key criterion in designing the perturbation is defining its objective, specifically, selecting the loss function that optimizes perturbations to ensure the LLM produces the correct answer while generating incorrect reasoning. The high-level intuition is to *make the CoT different yet preserve the same answer through optimization*. To achieve this, we formulate novel loss functions for our perturbations. We illustrate the basic loss instantiations using an example as shown below:

ICL (n_1): A list of (Q, CoT, A).
Q (n_2): If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?

A (n_3): The answer is 5.
CoT (n_4): Let's think step by step. There are 3 cars at first. 2 more cars arrive. 3 + 2 = 5.

The start location of the in-context example, current question, current answer, and current explanation are denoted as n_1, n_2, n_3, n_4, respectively. Further, we define an LLM as a function f that maps a sequence of tokens $x_{1:n}$ to an output logit distribution $f(x_{1:n})$, where tokens x_i belongs to a vocabulary of size V. In the following sections, we use logit distributions to help design the loss terms for $\mathcal{L}_c$ and $\mathcal{L}_a$ for the losses on CoT and answer part, respectively. Given a pair of original (reference) response x and perturbed response x', we have q_c, q'_c as the logits of $x_{n_3:n_4}$ (until CoT part), q_a, q'_a as the logits of $x_{1:n_3}$ (until answer part) (Eq. 1). If the logits are computed using the embedding space, then x is are embedding instead of set of tokens. Then, $\mathcal{L}_c$ and $\mathcal{L}_a$ are defined as the cross-entropy over the perturbed distribution compared with the reference distribution.

$$q_c, q'_c \leftarrow f(x_{n_3:n_4}), f(x'_{n_3:n_4});\ q_a, q'_a \leftarrow f(x_{1:n_3}), f(x'_{1:n_3}) \tag{1}$$

$$\mathcal{L}_c = \mathcal{L}_{CE}(q'_c, q_c),\ \mathcal{L}_a = \mathcal{L}_{CE}(q'_a, q_a) \tag{2}$$

The optimization objective carefully balances two competing goals: altering the reasoning while preserving the answer. To achieve this, we combine the two loss terms, scaling them via a coefficient $\lambda = \frac{n_4 - n_3}{n_3}$ to give uniform importance to each token. $\mathcal{L}_{opt}$ encourages the model to *maximize* $\mathcal{L}_c$, driving divergence in the CoT, while simultaneously *minimizing* $\mathcal{L}_a$, ensuring the final answer remains intact; thus, precisely targeting the model's reasoning instead of the answer.

$$\mathcal{L}_{opt} = \mathcal{L}_c - \lambda \cdot \mathcal{L}_a \tag{3}$$

4.2 Token-Level Perturbations

We design our token-level perturbation strategy to maximize the success rate of MATCHA while maintaining controlled edits to the input. Rather than directly replacing existing tokens, which risks distorting the original question and degrading answer accuracy, we adopt a two-stage token insertion and replacement approach. This allows us to inject perturbations without overwriting semantically critical content, preserving the model's ability to produce the correct answer. We then refine these inserted tokens through gradient-guided replacement, identifying which modifications most effectively disrupt the reasoning while keeping the answer stable. Compared to existing methods like GCG, our approach is more targeted and effective, yielding a lower unattackable rate and higher success rates (see Table 3). This design reflects a core insight: disrupting reasoning does not require semantic collapse, just precise, minimal shifts that steer the model's internal trajectory off-course.

Stage 1: Random token insertion. We initiate our attack by introducing a set of randomly selected tokens into the original question at randomly selected positions, effectively simulating a perturbation to the input, which we denote as "_" in Example 4.2. These random tokens are an initial disturbance that subtly alters the original question while preserving its structure and intent. The insertions aim to create a minimally modified version of the question that can expose vulnerabilities in the model's reasoning or decision-making process. We denote the ratio of inserted tokens compared to the original question length $(n_2 - n_1)$ as a.

Stage 2: Gradient-informed token replacement. We formulate our perturbation as an optimization problem by replacing the inserted tokens that are important for *maximizing* $\mathcal{L}_c$ *while minimizing* $\mathcal{L}_a$, to induce incorrect reasoning with the correct answer. First, we generate y_p and r_p by passing the unlabeled question x_p into the LLM. If y_p equals y_{gt} (ground truth label), we then generate reference logits q_c, q_a (Eq.1). After that, we start the optimization by generating the perturbed logits q'_c, q'_a (Eq.1) by passing in the perturbed inputs x'. With Eq. 2, we calculate $\mathcal{L}_c$, $\mathcal{L}_a$ as well as $\mathcal{L}_{opt}$.

We compute the gradient at each one-hot token indicator e_x with a shape of $((n_2 - n_1) * a, |V|)$, which has a value of 1 at the current input id and 0 elsewhere. To identify promising replacement candidates for each inserted token position, we evaluate them via forward passes. Specifically, we approximate the effect of

replacing token x_i using its gradient, as shown in Eq. 4, where g_{tok} has a shape of $((n_2 - n_1) * a, |V|)$. We use this to measure the influence of each token on the reasoning/answer. However, just knowing the influence is not enough; we need to use g_{tok} to find the *right locations* for the replacements and *right replacements* for those selected locations.

$$g_{tok} = \nabla_{e_x}(-\mathcal{L}_{opt}) \tag{4}$$

To find a set of *right locations* for inserted tokens (in the question part $x_{n_1:n_2}$), we compute $g_{tok} \cdot e_x$ (the gradient of the current token at each position). We then select the top-k token positions (highlighted in yellow) with the highest gradient scores (i.e. the positions that are most important for encouraging answer-reasoning inconsistency), as shown in Eq. 5, where Q denotes the selected token indices for replacement.

$$Q = \text{TopK}(g_{tok} \cdot e_x, k) \tag{5}$$

To find the *replacement token* for each selected position $q \in Q$, we identify the replacement token by selecting the vocabulary index with the highest gradient magnitude (Eq. 6).

$$rank_{tok}[q] = \arg\max_{v \in [|V|]} g_{tok}[q, v] \tag{6}$$

Finally, we get the replacement token list $rank_{tok}$, where each element corresponds to the top-ranked replacement token for each promising position. Specifically, the token that is predicted (via gradient guidance) to most effectively perturb the model's reasoning while preserving the correct answer.

Q (n_2): If _ there are __ 3 cars __ in the _ parking lot and __ 2 more _ cars _ arrive, _ how many __ cars are _ in the _ parking lot?

For each token in $rank_{tok}$, we leverage an LLM judge with inputs of original/perturbed questions to detect whether the question has a similar semantic meaning after replacing the token (the judge template can be found in Table 13). If the meaning changes, then we do not replace that token. We continue iterating this replacement/judge process for j steps. Our final perturbed answer y'_p and CoT r'_p are generated with the final perturbed input x'_p. We check the reasoning correctness of r'_p by passing (r_p, r'_p) into another judging LLM (the judge template can be found in Table 12). Our perturbation is successful if y'_p is correct yet r'_p is judged wrong. We present our token-level perturbation procedure in Algorithm 2 and 3.

4.3 Embedding-Level Perturbations

Embedding-level perturbations provide a powerful lens into LLM vulnerabilities beyond the discrete token space Schwinn et al. (2024); Xhonneux et al. (2024). Thus, we leverage embedding-level perturbations to subtly distort the model's reasoning path while preserving answer correctness, exposing latent instabilities that are not easily revealed by token-level edits. The high-level idea is to perturb the embedding space of the input questions ($E_c[n_1 : n_2]$) imperceptibly, such that it does not change the output token mapping, leading to the wrong reasoning yet correct answer. Similar to token-level perturbations, we generate the predicted answer y_p, r_p, and check whether y_p equals y_{gt}. If so, we then generate target logits q_c, q_a using input embedding E_c. After that, we generate the perturbed logits q'_c, q'_a by passing in the perturbed embedding inputs for CoT and answer parts (E'_c and $E'_c[: n_3]$). Given step-size α, perturbation region ϵ (a percentage of the original embedding space), and using Eq. 3 to calculate $\mathcal{L}_{opt}$, we update the embedding by performing l_∞ perturbation as follows:

$$E_{tmp} \leftarrow E'_c + \alpha \cdot \epsilon \cdot \text{sign}(\nabla_{E'_c} \mathcal{L}_{opt}) \tag{7}$$

Then, we clamp E_{tmp} into the interval of $[E_c - \epsilon, E_c + \epsilon]$ and only update only $E'_c[n_1 : n_2]$ part of the embedding with E_{tmp}, since we only perturb the embedding space of the question part. We continue the iterations for the perturbation with a budget of j steps. After that, we generate the perturbed answer y'_p and CoT r'_p with perturbed input embedding E'_c. To know whether the perturbation is successful, we follow the same procedure as the token-level perturbations. Algorithm 1 illustrates the embedding-level perturbation procedure of one unlabeled question.